Xijing Wang (Thomas)

Team 6

Teammates: Dillon Ma, Mithun Roy, Leena Aljurashi

ILR03

Feb 19, 2026

Individual Process

This week, my primary contribution for System Demo 1 was designing the software architecture and communication contract for our hexapod dancing robot. Specifically, I developed the full communication protocol between the Raspberry Pi and ESP32, the layered pseudocode for the Raspberry Pi control software, and collaborated with Leena on the embedded FreeRTOS pseudocode for the ESP32 side.

System Architecture Overview

Our hexapod robot uses a two-processor architecture: a Raspberry Pi handles high-level control (inverse kinematics, motion planning, vision-based calibration), while an ESP32 handles real-time servo control via FreeRTOS. The two communicate over UART at 115200 baud. Figure 1 shows the overall software layer diagram.

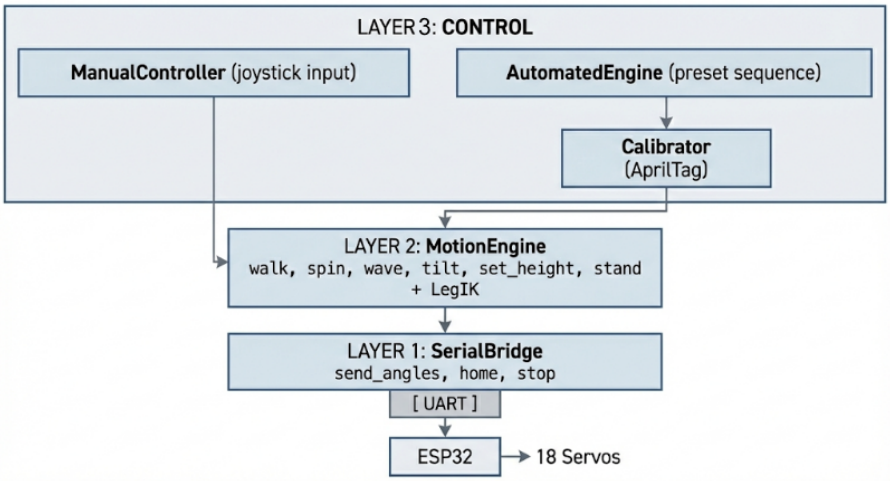

Figure 1: System software architecture. Layer 1 (SerialBridge) abstracts UART communication to the ESP32. Layer 2 (MotionEngine) provides motion primitives with inverse kinematics. Layer 3 (Control) offers both manual joystick and automated sequence modes, with AprilTag-based calibration.

Communication Contract Design

I designed the communication contract that defines the exact message format between the Raspberry Pi and ESP32. This contract ensures both processors have an unambiguous, shared understanding of every command and response. The protocol uses simple ASCII-based messages terminated by newline characters for easy parsing on the embedded side.

Table 1: Pi → ESP32 Commands

Command	Format	Description
Move	`M:a0,a1,...,a17\n`	Set all 18 servo angles (0–180°)
Home	`H:\n`	Return all servos to neutral (90°)
Stop	`S:\n`	Emergency stop, freeze all servos
Speed	`V:percent\n`	Set servo speed (0–100%)
Query	`Q:\n`	Request current status from ESP

Table 2: ESP32 → Pi Responses

Response	Format	Description
Acknowledge	`OK\n`	Command executed successfully
Error	`ERR:type\n`	Error (SERVO_FAULT, PARSE_FAIL, etc.)
Status	`ST:a0,a1,...,a17\n`	Current angles of all 18 servos
IMU	`IMU:r,p,y,ax,ay,az\n`	IMU sensor data (roll, pitch, yaw, accel)

The Move command encodes all 18 joint angles in a single comma-separated message, ordered as leg 1 (hip, knee, ankle) through leg 6. This flat ordering avoids parsing complexity on the ESP32 while maintaining a clear mapping to the physical robot. The system operates at 50 Hz update rate with angles constrained to 0–180°. Example messages:

```
Example Messages
Neutral stand:  M:90,90,90,90,90,90,90,90,90,90,90,90,90,90,90,90,90,90
Bow (front):    M:90,120,45,90,120,45,90,90,90,90,90,90,90,60,135,90,60,135
Leg 1 wave:     M:45,45,45,90,90,90,90,90,90,90,90,90,90,90,90,90,90,90
Home command:    H:
Emergency stop:  S:
```

Raspberry Pi Software Architecture

I designed the Raspberry Pi software as a four-layer architecture, where each layer has a single, well-defined responsibility. This layered approach allows us to develop and test each component independently.

Listing 1: Raspberry Pi Software Layers (Pseudocode)

```
# === LAYER 1: HARDWARE ABSTRACTION ===

procedure SerialBridge.send_angles(angles[18]):
    Format angles as "M:a0,a1,...,a17\n"
    Transmit over UART
    Wait for OK or ERR response

procedure AprilTagTracker.get_robot_pose():
    Capture camera frame, detect AprilTag
    return Pose(x, y, z, yaw) or None

# === LAYER 2: MOTION ENGINE ===

```

```
procedure MotionEngine.walk(dir_x, dir_y, steps):
    Tripod gait: alternate legs {1,4,5} and {2,3,6}
    for each step:
        Compute foot target positions
        IK solve all 6 legs -> 18 joint angles
        SerialBridge.send_angles(angles)

procedure MotionEngine.spin(angle):
    Rotate body in place using tripod gait
    IK solve all 6 legs per phase, send angles

procedure MotionEngine.wave_leg(leg_id, amplitude):
    Raise one leg and wave while others stand
    IK solve, send angles

procedure MotionEngine.set_body_height(height):
    Adjust all 6 feet z-position to change body height
    IK solve, send angles

procedure MotionEngine.stand():
    Set all feet to default standing position
    IK solve, send angles

# === LAYER 3: CALIBRATION ===

procedure Calibrator.full_calibration():
    Execute test walks, measure error via AprilTag
    Execute test spins, measure error via AprilTag
    return correction_factors(walk, rotation)

# === LAYER 4: CONTROL MODES ===

procedure ManualController.run():
    while active do:
        Read joystick input
        left stick  -> engine.walk(x, y, 1)
        right stick -> engine.spin(angle)
        triggers    -> engine.set_body_height(h)
        bumpers     -> engine.set_body_tilt(roll, pitch)
        d-pad up    -> engine.wave_leg(1, 30)
        select      -> engine.stand() + stop

procedure AutomatedEngine.execute_sequence(sequence, config):
    for each (timestamp, move_type, params) in sequence:
        Apply calibration correction to params
        Call corresponding MotionEngine function
```

ESP32 Embedded Software (FreeRTOS)

I also collaborated with Leena on the FreeRTOS task architecture for the ESP32. Leena is primarily responsible for the ESP32 embedded code, and together we defined the task structure and priorities. The ESP32 runs six concurrent tasks with assigned priorities to ensure real-time servo control while handling Bluetooth commands, vision data, and IMU readings. The highest-priority SafetyMonitorTask provides emergency stop capability if communication is lost or the robot tilts beyond safe limits.

Listing 2: ESP32 FreeRTOS Task Architecture (Pseudocode)

```
procedure APP_MAIN:
    Initialize GPIO, IMU (I2C), UART, Bluetooth, servo PWM
    Initialize shared data structures and inter-task queues
    Create BluetoothRxTask     (priority: high)
    Create VisionRxTask        (priority: medium)
    Create ImuTask             (priority: medium-high)
    Create ControlTask         (priority: high, periodic)
    Create ServoOutputTask     (priority: high)
    Create SafetyMonitorTask   (priority: highest)
    Start FreeRTOS scheduler

procedure ControlTask:           # Periodic at 100 Hz
    while true do:
        Read ManualCommand, BodyState, VisionPose snapshots
        if mode == manual then
            Set desired (v, w) from joystick commands
        else
            Set desired (v, w) from vision / planner
        end if
        Apply stabilization using IMU (tilt limits)
        Compute gait phase and foot trajectories
        IK solve -> target joint angles
        Send angles to ServoCommandQueue

procedure SafetyMonitorTask:     # Highest priority
    while true do:
        if Bluetooth timeout exceeded then
            Set desired motion to STOP
        end if
        if tilt exceeds safety threshold then
            Trigger emergency stop (freeze / neutral)
        end if
        Sleep 20-50 ms
```

Challenges

One challenge I faced was defining the right level of abstraction for the communication protocol. The initial design included complex nested message types, but after discussion with the team I simplified it to flat ASCII strings that are easy to parse on the ESP32's limited resources. Balancing expressiveness (supporting multiple command types, error reporting, and status queries) with simplicity (fast parsing in a real-time embedded context) required several iterations.

Another challenge was ensuring the layered software architecture remained clean while supporting two fundamentally different control modes (manual joystick and automated choreography). The key insight was to make both modes call the same MotionEngine layer, so that motion primitives like walk, spin, and wave are shared and only the decision-making differs between layers.

Team Work

Our team divided responsibilities across mechanical design, software architecture, and system integration. I focused on the software architecture, communication contract, and Raspberry Pi pseudocode. Leena is responsible for the ESP32 embedded software (FreeRTOS), and I collaborated closely with her to ensure the communication contract and task architecture were consistent between the Pi and ESP32 sides. Mithun worked on the hardware side, including motor/servo selection and the power supply diagram. Dillon worked on the CAD design of the hexapod robot in SolidWorks, modeling the body, leg linkages (coxa, femur, tibia), and full assembly.

For the system demo, we presented Dillon's detailed CAD model showing the hexapod base, top plate, and six three-DOF legs, along with the software architecture and communication design described above.

Plans

Our team goals for the next milestone (2/25) include finalizing leg dimensions and building prototype legs, testing the physical mockup control architecture over Bluetooth, completing power system demo #1, identifying the final motor, and updating the Gantt chart. On my side, I plan to:

- Implement the communication contract in working Python (Pi side) and C (ESP32 side) code and validate message round-trips over UART.
- Work with Dillon to finalize leg dimensions and incorporate them into the inverse kinematics solver for the three-joint leg (coxa, femur, tibia).
- Build and test the physical mockup control architecture, specifically testing Bluetooth communication between the controller and the ESP32.
- Continue collaborating with Leena on the FreeRTOS implementation and with Mithun and Dillon as hardware and mechanical parts are fabricated.